Arthur Chevallier

Designing_Digital_Twins_for_Progressive_Fidelity__A_Layer...

LIST Luxemburg Institute of Science and Technology

Document Details

Submission ID
trn:oid:::3117:617339485

Submission Date
Aug 26, 2026, 2:41 PM GMT+2

Download Date
Aug 26, 2026, 2:50 PM GMT+2

File Name
Designing_Digital_Twins_for_Progressive_Fidelity__A_Layered_Architecture_for_Airport_Operatio....pdf

File Size
1.1 MB

16 Pages

5,647 Words

33,526 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

35 Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks

3 Missing Quotations 1%
Matches that are still very similar to source material

1 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
13% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **35** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **3** Missing Quotations 1%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
13% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		2%
2	Publication	
Vaishnav Arun Bhosale. "The Digital Twin in Aviation: A Review of Core Technolog...		<1%
3	Internet	
mdpi-res.com		<1%
4	Internet	
ouci.dntb.gov.ua		<1%
5	Internet	
www.springerprofessional.de		<1%
6	Internet	
dokumen.pub		<1%
7	Internet	
pdffox.com		<1%
8	Internet	
www.mdpi.com		<1%
9	Internet	
openrepository.aut.ac.nz		<1%
10	Internet	
www.scirp.org		<1%

11	Internet	
research.sabanciuniv.edu		<1%
12	Internet	
www.alexandria.unisg.ch		<1%
13	Internet	
kipdf.com		<1%
14	Internet	
listens.online		<1%
15	Internet	
ntnuopen.ntnu.no		<1%
16	Publication	
"Multidisciplinary Social Networks Research", Springer Science and Business Med...		<1%
17	Internet	
export.arxiv.org		<1%
18	Internet	
pure.tue.nl		<1%
19	Internet	
www.diagnostyka.net.pl		<1%
20	Internet	
www.medintensiva.org		<1%
21	Publication	
Mustafa Ibrahim Khaleel. "A digital twin-driven hybrid intelligence framework for...		<1%
22	Internet	
archive.org		<1%
23	Internet	
dspace.mit.edu		<1%
24	Publication	
Massimiliano Masi, Rosario Pugliese, Francesco Tiezzi. "Security Analysis of Stand...		<1%

25 Internet

aos.ro <1%

26 Publication

"HCI for Cybersecurity, Privacy and Trust", Springer Science and Business Media L... <1%

27 Internet

vireo.cs.cityu.edu.hk <1%

28 Publication

Ion Turcanu, Oussema Gharsallaoui, Patrick Gratz, Maxime Poncin, German Casti... <1%

29 Internet

construpedia.ai <1%

30 Internet

www.ama-assn.org <1%

31 Internet

ijerece.com <1%

32 Internet

libweb.kpfu.ru <1%

33 Publication

Jun Wen, Metin Kozak, Joshua Aston, Wei Wang. "Interdisciplinary Research Touri... <1%

A Layered Architecture for Digital Twins with Progressive Fidelity: A Case Study in Airport Operations Modelling with AI-Assisted Development

Arthur Chevallier and German Castignani

Luxembourg Institute of Science and Technology
arthur.chevallier@list.lu, german.castignani@list.lu

Abstract. Digital twins of complex cyber-physical systems often need to evolve from simple representations towards increasingly higher fidelity, integrating physical models, geospatial information, live data, and decision support. However, existing digital-twin architectures do not explicitly address how such increases in fidelity can be introduced while preserving previously implemented functionality and interfaces. We investigate a layered fidelity architecture in which higher-fidelity capabilities extend lower layers through their existing contracts rather than requiring their restructuring. To support implementation of this architecture, we use a specification-first methodology in which domain knowledge, service contracts, and reusable development patterns are captured in machine-readable specifications and skill files accessible to both human developers and large language model agents. We instantiate the approach in VirtualSky International Airport (EVSK), a fully open-source airport digital twin spanning flight operations, passenger flows, baggage handling, weather impacts, incident cascades, cost analysis, and planning across nine microservices. Based on more than fifty development sprints and commit-level analysis of the implementation, we examine whether the architecture supports progressive fidelity and whether specification-first, agent-assisted development can preserve its contracts as capabilities are added. The results provide evidence for a practical architectural principle for evolving digital twins, together with lessons on the benefits and limitations of AI-assisted development in a long-running, multi-service system.

Keywords: Digital twins · Airport operations · Cyber-physical systems · AI-assisted software engineering · Specification-driven development · Simulation architecture · Progressive fidelity.

1 Introduction

Digital twins are virtual counterparts of physical systems maintained through structured data flows [1,2]. In practice, they rarely remain at a fixed level of detail. A project may begin with an event-level or logical model and later acquire

iThenticate Page 7 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

physical simulation, geospatial representation, live data integration, analytics, and decision support [3]. Deployed systems increasingly combine these capabilities within integrated architectures [4]. This evolution creates a problem that is architectural rather than merely functional: increasing fidelity may require changes to representations, interfaces, and data flows that earlier capabilities already depend on. As a result, adding fidelity can lead to restructuring rather than straightforward extension, making long-term evolution and reproducibility difficult [5].

Existing digital-twin frameworks provide important architectural decompositions. ISO 23247 defines a reference architecture for manufacturing digital twins [6], RAMI 4.0 organises industrial systems across multiple dimensions [7], and other frameworks separate concerns ranging from data acquisition to emulation [8]. These approaches establish layers and components, but do not explicitly treat *fidelity-preserving evolution* as an architectural constraint: namely, that higher-fidelity capabilities should extend an existing twin through the contracts of lower layers rather than requiring those layers to be restructured. This is the gap investigated in this work.

We therefore ask:

RQ1: How can a digital-twin architecture support progressive increases in fidelity while preserving the contracts and functionality of previously implemented layers?

We investigate this question through VirtualSky International Airport (EVSK), a fully synthetic airport digital twin. Airports provide a demanding testbed because aircraft, vehicles, passengers, baggage, infrastructure, schedules, weather, and disruptions interact across multiple operational timescales. Existing research provides mature models for individual aspects of this system, including delay propagation [9], delay cost [10], passenger-flow prediction [11,12], and airport digital twins [13,14]. EVSK instead uses the airport as an instantiation in which progressively richer representations can be implemented within a common architecture. The system comprises approximately 45,000 lines of Python across nine microservices and approximately 28,000 lines of React/TypeScript dashboards.

The second question concerns how such an architecture can be implemented and maintained. AI-assisted software engineering has demonstrated strong capabilities in code generation and repository-level development [15,16,17,18]. However, local correctness does not guarantee architectural consistency across a distributed system. Event schemas, shared concepts, service boundaries, and data flows must remain coherent as the system evolves. We therefore use a specification-first methodology in which service specifications, event schemas, data models, and simulation rules are defined before implementation, supplemented by reusable "skill files" encoding implementation patterns and known pitfalls for coding agents. This motivates:

RQ2: To what extent can specification-first, agent-facing development preserve architectural contracts during sustained implementation of a progressively evolving digital twin?

iThenticate Page 7 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 8 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

The paper makes four contributions:

1. a layered fidelity architecture that treats progressive, contract-preserving evolution as an explicit architectural principle;
2. an empirical examination of this principle through EVSK, including commit-level analysis of changes between fidelity layers;
3. a specification-first, agent-facing development methodology for maintaining architectural contracts during implementation; and
4. an account of the benefits, limitations, and failure modes observed across more than fifty development sprints.

The contribution to digital-twin research is therefore the architectural principle and its empirical examination, rather than the airport application itself. The contribution to AI-assisted software engineering is evidence about using explicit architectural specifications to constrain agent-assisted development of a long-lived, multi-service system.

The remainder of the paper is organised as follows. Section 2 discusses related work. Section 3 presents the layered fidelity architecture, Section 4 describes the development methodology, and Section 5 presents EVSK. Section 6 reports the evaluation, followed by discussion and conclusions in Section 7.

2 Related Work

Digital-twin research provides several foundations for the architecture considered here. The concept originated as a virtual counterpart of a physical system [2] and was subsequently formalised as a lifecycle-spanning computational representation [1]. Kritzinger et al. [19] distinguish digital models, digital shadows, and digital twins according to the degree and direction of automated data exchange. Reference architectures such as ISO 23247 [6], RAMI 4.0 [7], and the architecture proposed by Redelinghuys et al. [8] provide different forms of decomposition into entities, layers, and services. Reviews nevertheless identify extensibility, interoperability, and reproducibility as continuing challenges [5,20]. Our work addresses a narrower question within this literature: how to make increasing fidelity an explicit, contract-preserving evolution mechanism rather than simply a collection of architectural layers.

Airport modelling provides the application context for examining this question. Analytical and simulation-based research has addressed delay propagation [9] and the cost of delay [10], while machine learning has been applied to passenger-flow and boarding-time prediction [11,12]. More recent work explicitly applies digital-twin concepts to airport operations, including turnaround operations [13], landside simulation [14], and airport-wide adoption guidance [21]. These studies demonstrate the feasibility and value of airport digital twins, but generally address particular operational scopes or fixed capabilities rather than the problem of evolving a twin across fidelity levels without restructuring previously implemented functionality. EVSK uses the domain as a demanding instantiation of the architectural principle rather than as the primary research contribution.

iThenticate Page 8 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 9 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

Finally, AI-assisted software engineering has progressed from code synthesis [15] and controlled productivity studies [16] to repository-level benchmarks such as SWE-bench [17] and agent-based development environments [18]. Approaches such as MetaGPT further demonstrate the value of explicit software-development procedures for coordinating agents [22]. These studies primarily evaluate local task completion or repository-level issue resolution. They provide less evidence about maintaining architectural consistency across a long-running, distributed system as new capabilities are added. Our work investigates whether specification-first development can provide such a constraint, using the progressive-fidelity architecture as the concrete setting.

The resulting gap is therefore the combination of two problems: *how to evolve a digital twin across fidelity levels without restructuring existing layers, and how to maintain the resulting architectural contracts during sustained agent-assisted development*. RQ1 addresses the architectural problem; RQ2 addresses the engineering process used to implement it.

3 Layered Fidelity Architecture

The architecture is a stack of four capability layers: a logical event-driven simulation, a physical layout, a geospatial and real-world data layer, and a decision-support layer. Each layer consumes only the contracts exposed by the layer beneath it, and nothing above a layer is visible to it. Fidelity is acquired by accretion rather than rewrite. Thus, a project can start at the bottom of the stack and add capability over its lifetime without disturbing what already works.

This pattern is not specific to aviation. A hospital campus twin could be built the same way: patient, staff, and equipment state machines communicating over an event bus at the base, ward and building layout added on top of that, real-time location tracking and environmental sensors added next, and bed-allocation or staffing optimisation sitting at the top. A factory floor, a shipping port, or a district energy grid could be layered the same way. What the architecture fixes is the order in which fidelity is added and the contracts between layers, not the domain those layers describe. We use an airport, EVSK, as the concrete instantiation and stress test for the rest of this paper, but the architecture itself is not tied to flights, gates, or passengers.

The four layers, described independently of any particular domain, are as follows:

- **L0, logical event-driven simulation.** Domain entities are represented as state machines that communicate purely through events. There is no geometry and no external data at this layer. Something happens, an event records it, and downstream consumers react.
- **L1, physical layer.** Spatial layout is added on top of L0. Positions, distances, and the travel or transfer times they induce between entities already defined at L0.

iThenticate Page 9 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 10 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

– **L2, geospatial and real-world data layer.** The twin is connected to the outside world. Map-based visualisation, live external feeds, and reference data ground the simulation's logical and physical state in observed reality.
– **L3, decisional layer.** Forecasting, scenario analysis, and recommendation sit on top of the first three layers. Autonomous actions reach the source system only through approval gates.

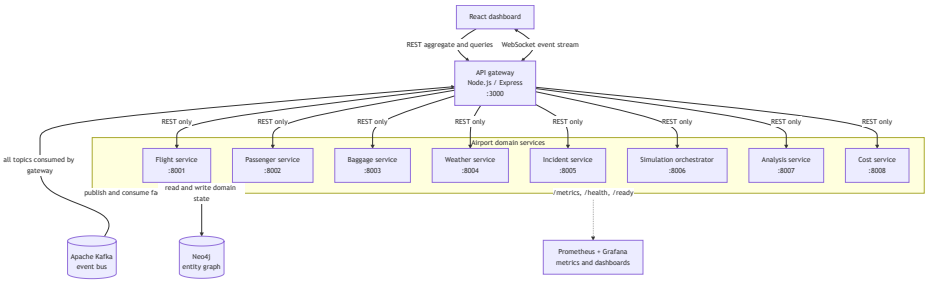


Fig. 1. EVSK system architecture. The gateway is the synchronous boundary for clients; Kafka decouples domain services, while Neo4j stores the operational graph.

From L0 to L2, we create a digital shadow of the use-case by representing the airport through data and dashboards. Flow goes from the open source system into the twin. We introduce L3 to close the loop by acting as the command gate.

A separate design axis governs how a single layer matures, independent of which layer it is. Capability within a layer deepens along a fixed ladder, from visualisation, to monitoring and alerting, to prediction, to prescription, to actionability. A layer counts as mature once it has climbed this ladder, not merely because it renders something.

Two properties keep the stack composable as it grows. Each layer depends only on the layer directly below it, so a layer can be upgraded, replaced, or reimplemented without touching the layers above or below it. And the dependency direction is enforced strictly one way: L3 may read from L0–L2, but nothing below L3 ever imports from it, which is what keeps the decisional layer removable without destabilising the shadow underneath it.

Section 5 shows how EVSK realises each of these layers with a concrete technology stack, together with the architectural invariants that keep them composable in practice.

4 Enforcing Layer Contracts with Specification-First AI-Agent Development

The layered architecture in Section 3 keeps fidelity additions from forcing a rewrite by fixing what each layer may depend on. That guarantee holds only if

iThenticate Page 10 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 11 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

the code an agent writes actually respects those contracts, and nothing about an LLM agent enforces that on its own. The methodology that keeps the two aligned rests on an inversion of the usual workflow: the specification is written first, is machine-readable, and is authoritative. Implementation may not deviate from it, and where reality demands a change, the spec is updated first and the code follows. Before any service is implemented, four definitions already exist: the event-bus schemas, the graph data model, the simulation rules, and one specification per service describing its endpoints, events, state machines, and ownership boundaries. Each new layer, in practice, starts as a specification of the contract it will expose to the layer above and consume from the layer below, before a single line of that layer's code is written.

This discipline has an intellectual lineage older than language models. Writing the authoritative description before the implementation echoes formal specification traditions [23], and decomposing the system so that each service hides exactly one set of design decisions follows Parnas' information-hiding criteria [24]. What is new is the consumer. Human developers treat specifications as documentation that drifts from the code; an LLM agent treats them as ground truth to be followed literally. A stale spec once mildly misled human readers, but now it actively misdirects the agent that writes the next change.

Complementing the per-service and per-layer specifications are "skill files", which encode reusable project knowledge in a form readable by both humans and agents. Each skill file has three parts: concept explanation, copy-paste patterns, and gotchas. The patterns matter because code generation is imitation, and an agent grounded in the project's idiomatic patterns produces consistent code where one left to its prior defaults produces whatever its training data favoured. The gotchas encode failures that review caught the hard way, so the same mistake is prevented from recurring. Cross-cutting skills exist for exactly the four architectural invariants from Section 3: the simulation clock, the event bus, the graph as single source of truth, and the no-silent-mutation rule. These are the invariants an agent is most likely to violate under time pressure, since violating any one of them is often the fastest way to make a feature appear to work. Each service additionally owns one skill file documenting its own state machines and quirks.

Routing makes the difference between artefacts that exist and artefacts that get used. Agent entry points, `CLAUDE.md`, `AGENTS.md`, and editor rule files, instruct the agent to load the relevant specification and skill file before writing code, so authoritative context arrives by default rather than by request. Hard rules function as guardrails against known LLM failure modes, and each one maps directly onto a layer contract rather than a general best practice: no wall-clock time in business logic, no service-to-service HTTP calls, no silent mutations, and all data synthetic. Because each rule corresponds to a specific invariant, a violation is detectable by search or contract test the moment it happens, rather than surfacing later as a layer-boundary failure that looks unrelated to its cause.

iThenticate Page 11 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 12 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

The loop closes through sprint post-mortems. Defects and near-misses found in review, including layer-boundary violations that slipped past the hard rules, are promoted back into skill files as gotchas. The knowledge base is thereby living, shared between human and machine readers, and accumulates detail exactly where an added layer most recently threatened the contract of the layer beneath it. Therefore, progressive fidelity has to be found twice: once in what the twin models, and again in how faithfully the implementation matches what was specified.

5 EVSK: Instantiating the Layered Architecture

EVSK instantiates the four-layer architecture of Section 3 as a working system comprising roughly 45,000 lines of Python and roughly 28,000 lines of React/TypeScript dashboards. The stack is deliberately conventional—nine FastAPI domain services behind a thin Node.js/Express API gateway, a React/TypeScript dashboard suite, Kafka as the event backbone [25], Neo4j as the graph store [26], Prometheus/Grafana/Loki/Jaeger for observability, and Kubernetes manifests with a staged CI pipeline—so that what the system tests is the architectural claim, not an exotic technology choice.

Each layer is realised with a concrete slice of this stack. L0 uses Kafka as the event bus and Neo4j as the graph store, holding flights, passengers, baggage, weather, and incidents as domain state machines. L1 adds terminal layouts, gate positions, and runway and taxiway geometry. L2 adds Mapbox/Cesium visualisation, an ADS-B flight overlay, live and historical METAR weather, BTS schedule calibration, and OurAirports/OpenFlights reference data. L3 adds forecasting, what-if and Monte-Carlo scenario analysis, anomaly detection, operational recommendations, and discounted-cash-flow investment appraisal on Eurocontrol Standard Inputs [27], detailed further below.

A set of architectural invariants, general to the stack rather than specific to EVSK, keeps these layers composable. No service reads wall-clock time directly; a single orchestrator broadcasts ticks on a dedicated topic, and all time-based logic reacts to those ticks rather than to the system clock. In EVSK this runs at configurable speed up to 3600× real time, which also gives deterministic replay for free. The event bus carries facts only. Events record what happened, never what another service should do next, so consumers remain independent of producers and the log doubles as an audit trail. The graph store is the single source of truth for structural state. Each service is the sole writer of its own node labels, and any in-memory cache is derived state, rebuildable from the graph after a restart. Every state change emits an event, so the graph, the dashboards, and the analytical layers cannot drift apart from one another silently.

EVSK's own build history is evidence for this composability: the system moved from pure simulation, through map and live-data integration, to forecasting and financial planning, without re-platforming any earlier layer. Section 6 audits this claim directly against the commit history.

iThenticate Page 12 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 13 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

Domain depth is where the implementation earns the word "twin". The flight domain runs a nine-state lifecycle state machine with runway queueing and delay cascades that propagate up to five hops, mirroring network-level propagation models [9] (see Annex, Fig. 5). Passenger flow feeds a LightGBM model that predicts security-queue depth ninety minutes ahead [11,12]. Baggage travels a thirty-zone conveyor network with dangerous-goods screening, and weather is a four-state finite-state machine whose runway-capacity impacts cascade into the flight domain (see Annex, Fig. 6). A rule-based incident engine cascades five hazard types through the graph under explicit depth limits.

The digital twin is intentionally open at both its configuration and data boundaries. Airport identity, terminals, gates, runways, operational constants, and cascade limits are loaded from YAML rather than embedded in service logic; changing the airport therefore changes configuration, not event contracts or domain algorithms. A shared data-source registry exposes the same adapter interface to simulated, historical, and live providers, so new sources can be added without changing downstream consumers. This is the practical Open/Closed property of the twin: extend the represented world by adding configuration, fixtures, or adapters, while keeping the tested core closed to modification.

The simulated day is operationally plausible at scale: 420 flight movements, roughly 30,000 passengers, and roughly 36,000 baggage items, driven by the simulation clock described above.

Crucially, the decisional layer is realised in production code rather than promised in the future tense. A financial service maintains profit-and-loss statements, passenger-rights liability under EC 261/2004 [28], and carbon accounting. A planning service runs a Monte-Carlo scenario engine for infrastructure decisions, producing NPV, IRR, and payback appraisals grounded in Eurocontrol Standard Inputs [27]. An analysis service provides anomaly detection via an Isolation Forest over the operational state, slot-allocation MILP, and an autonomous mode whose recommendations enter an approval queue, classified as auto-executable, human-gated, safety-guarded, or operator-blocked, and reach other domains only through per-domain command topics. This is the mechanism by which L3 closes the loop: commands are first-class, gated, and auditable, not advisory text on a dashboard.

Reproducibility is structural rather than aspirational. All specifications, skill files, infrastructure definitions, and data pipelines are open source at https://github.com/arthur-chevallier/airport-digital-twin (DOI: 10.5281/zenodo.22065741), and the airport's identity lives in configuration rather than code—a different airport is a YAML file, not a fork.

6 EVSK: Evaluation of progressive fidelity in our use-case

This section demonstrates progressive fidelity as it played out in EVSK. The repository does not preserve a formal sprint or milestone ledger, so evidence here is based on commit history, file structure, and screenshots rather than a quantitative benchmark.

iThenticate Page 13 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 14 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

6.1 Progressive fidelity, layer by layer

Figures 2 to 4 walk the system from its logical core to its decisional layer. Figure 2 shows the Neo4j graph that serves as L0's single source of truth, populated purely from simulated events. The physical layer (L1) defines the terminal and gate geometries using standardized GeoJSON fixtures. Figure 3 shows the L2 data-source connector aggregating simulated and real feeds behind one adapter interface. Figure 4 shows the cost dashboard, the first decisional (L3) capability, built entirely from events published by the layers beneath it.

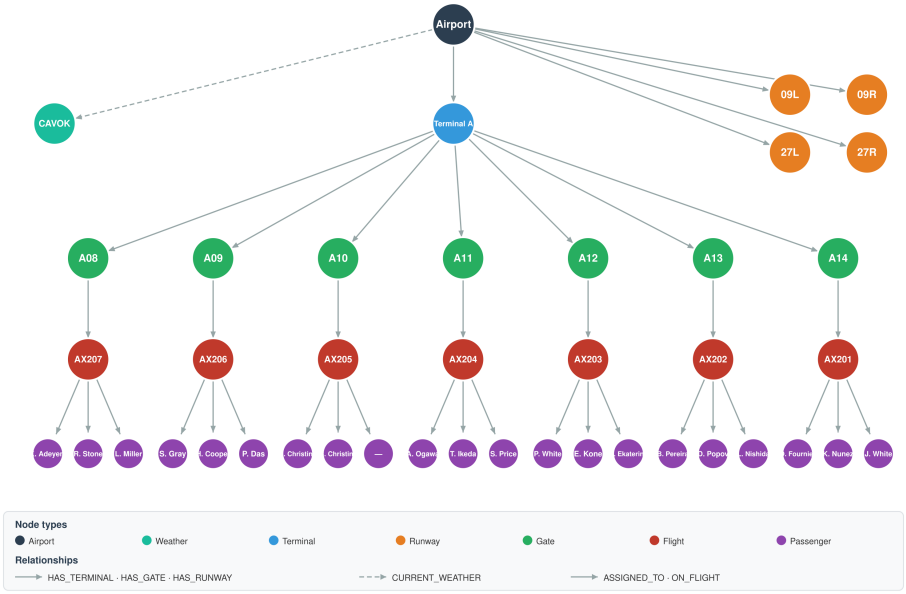


Fig. 2. L0: Neo4j as source of truth and context broker.

These screenshots show what each layer looks like once built, not what happened underneath as it was added, so we checked the commit history directly. Five of the largest refactor and feature commits (`23bea42`, `8536eab`, `10536f1`, `4d8e4b4`, `22c5c18`) each modified between 41 and 94 files across four or more services at once. Two of these, `23bea42` and `22c5c18`, also changed L0's event-bus and graph-model documentation, so we inspected both line by line.

The clearest example is the introduction of the cost service in `22c5c18`. Adding it meant extending the event bus with a new topic, `cost.events`, and a new event type, `CostRecorded`; extending the graph model with a new node type, `CostRecord`, its relationships, and its indexes; and widening the consumer list of four existing topics (`flights.events`, `passengers.events`, `baggage.events`, `incidents.events`) so the new service could read them. None of this changed

iThenticate Page 14 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

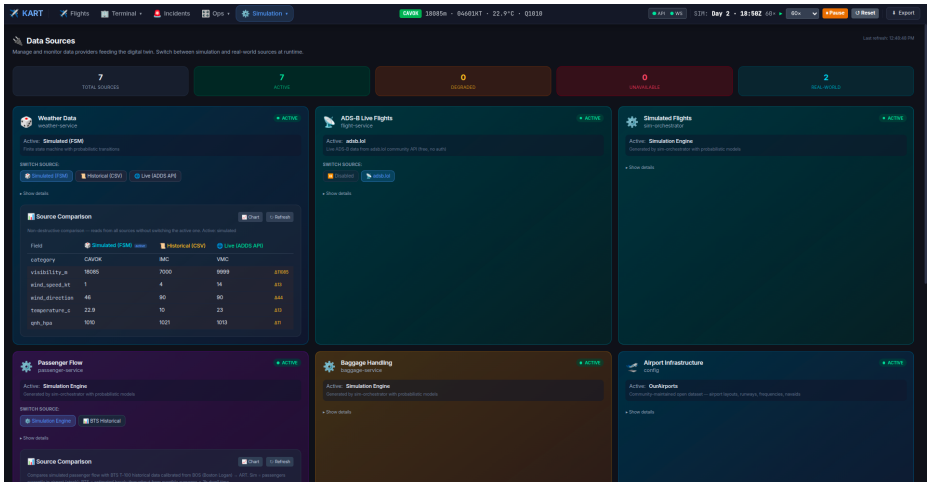


Fig. 3. L2: data source connector aggregating simulated and real data flows used by the simulation.

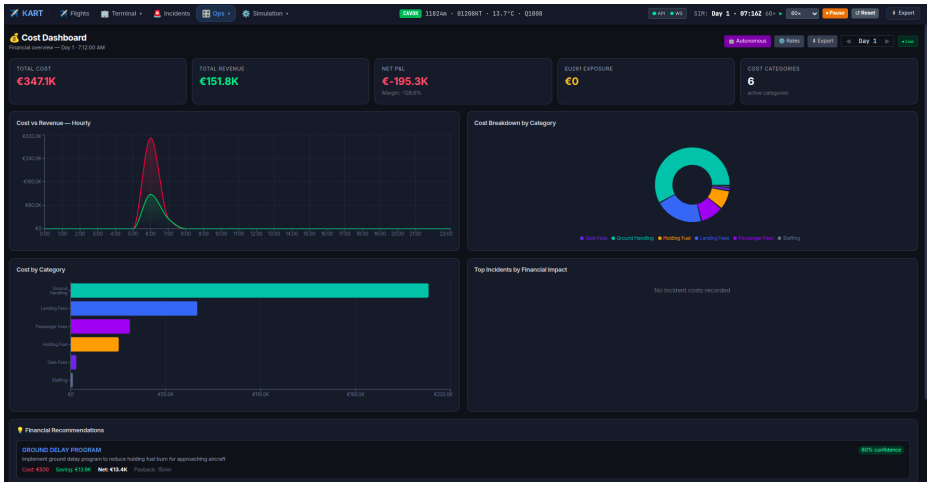


Fig. 4. L3: cost dashboard, the first decisional-layer capability, built by consuming events from the layers below.

iThenticate Page 16 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

what an existing event carried or how an existing consumer parsed it. The same pattern holds in `23bea42`, which added command topics for flights, passengers, and baggage alongside new event types, again without reshaping anything that already existed. Where existing dispatch code was touched, for instance cost-service's handler being extracted into a shared `process_envelope` function with added idempotency tracking, the same event cases and topic subscriptions were retained throughout.

This is what we mean by progressive fidelity in practice. Adding a new service, and even refactoring code around it, is expected as the system grows; what matters is whether the core contracts underneath stay intact. Across the commits audited here, the skills and event contracts a service already depended on stayed the same. We extended capability, we did not rewrite what was already working.

6.2 Specification and skill file density

The repository contains 90 commits on `main`, ten service directories (nine domain services + the API gateway) comprising 109 files, nine per-service specification files (`SPEC.md`), seven cross-cutting skill files, eight architecture documents, and two agent-facing entry points (`AGENTS.md`, `CLAUDE.md`). This density supports the specification-first discipline argued for in Section 4: the two commits examined above that extended the event contract also updated the corresponding architecture documents in the same commit, `EVENT_BUS.md` in both cases and `DATA_MODEL.md` and `docs/services/cost-service/SPEC.md` in `22c5c18`, rather than leaving code and specification to drift apart.

A mid-project audit also found cases where documentation and running code had drifted, discussed alongside its remediation in Section 7.

7 Discussion and Conclusion

The results address the two research questions directly. **RQ1** asked whether fidelity can be increased while preserving previously implemented layers. EVSK provides evidence that it can: new capabilities were added by extending existing contracts rather than restructuring lower layers. Configuration-driven airport identity, shared Kafka infrastructure, and hexagonal boundaries helped keep these interfaces stable as the system grew. The key result is therefore not the number of layers itself, but the use of explicit contracts to make fidelity an evolutionary property of the architecture.

RQ2 asked whether specification-first, agent-facing development can preserve these contracts during sustained implementation. The experience was positive but not sufficient on its own. A mid-project audit found boundary failures including diverging duplicated constants, recommendations that were not connected to command topics, and an ML endpoint that reported success without having trained a model. These failures show that specifications constrain intended behaviour but do not guarantee implementation conformance. Periodic audits and mutation-style guards [29] were therefore necessary complements.

iThenticate Page 16 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 17 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

The two findings are complementary: *the architecture defines what must remain stable; the methodology helps developers and agents preserve it.* Neither is sufficient alone. A layered architecture can drift without verification, while a detailed specification can faithfully encode an incorrect boundary. The practical lesson from EVSK is that explicit contracts, specifications, and systematic verification should be treated as a single development discipline.

The approach is not specific to airports. Other cyber-physical domains with progressively increasing fidelity, such as ports, hospitals, and warehouses, could use the same pattern. However, this study is a single-case investigation with a small team, one domain, and synthetic ground truth [30]. Learning effects are also confounded with the effects of specification-first and agent-assisted development, so the observed productivity and defect trends should not be assumed to generalise.

Future work includes completing command-topic coverage for decisional capabilities, using EVSK's deterministic event history as a benchmark for multi-agent reinforcement learning, and extending the configuration-driven architecture to federated multi-airport studies.

Overall, EVSK supports **RQ1** by demonstrating the feasibility of contract-preserving progressive fidelity, and supports **RQ2** by showing that specification-first development can help maintain those contracts during agent-assisted development, while requiring independent verification. The broader contribution is an architectural and development pattern for growing complex digital twins without making each increase in fidelity a reason to rebuild the layers beneath it.

Acknowledgments and Use of AI

20 This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. All computation, development, and infrastructure for the EVSK implementation were carried out on the authors' own resources, without external infrastructure support. The authors thank colleagues 32 at the Luxembourg Institute of Science and Technology for their review of and feedback on earlier drafts of this manuscript.

Artificial intelligence tools were used to support the writing and editing of this manuscript, including text rephrasing, proofreading, and improving clarity. All scientific content, interpretations, and conclusions were produced and validated by the authors, who 26 takes full responsibility for the accuracy and integrity of all statements in the paper.

iThenticate Page 17 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 18 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

Annex

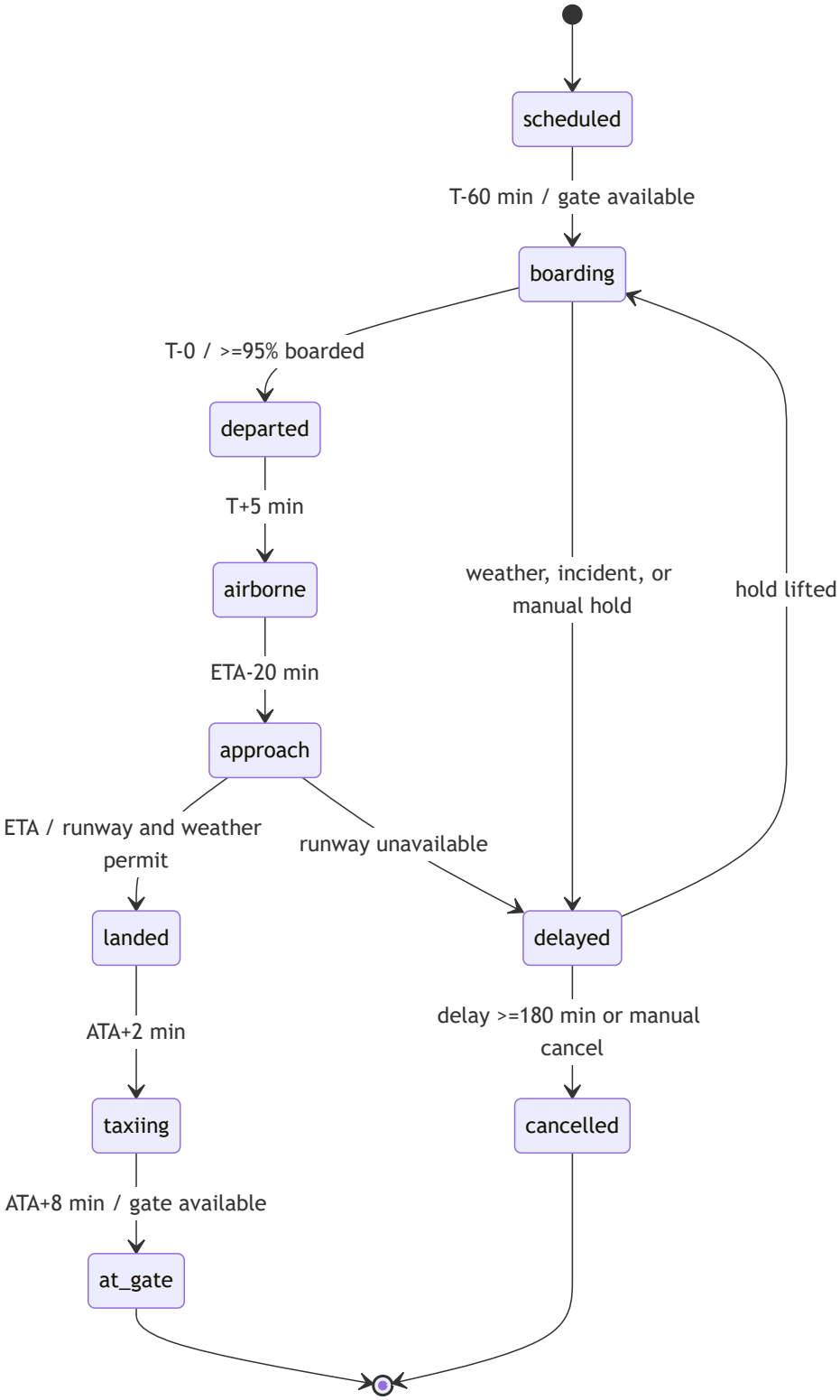


Fig. 5. Flight lifecycle driven by simulated time and constrained by gates, runways, weather, incidents, and boarding progress.

iThenticate Page 18 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 19 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

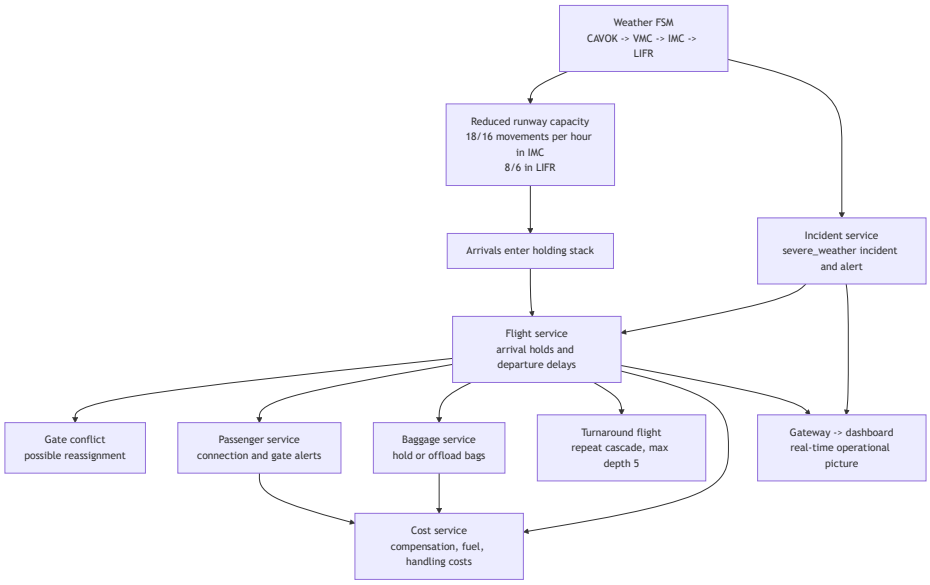


Fig. 6. Causal propagation from weather degradation to runway capacity, flight delays, passenger and baggage disruption, and operational cost.

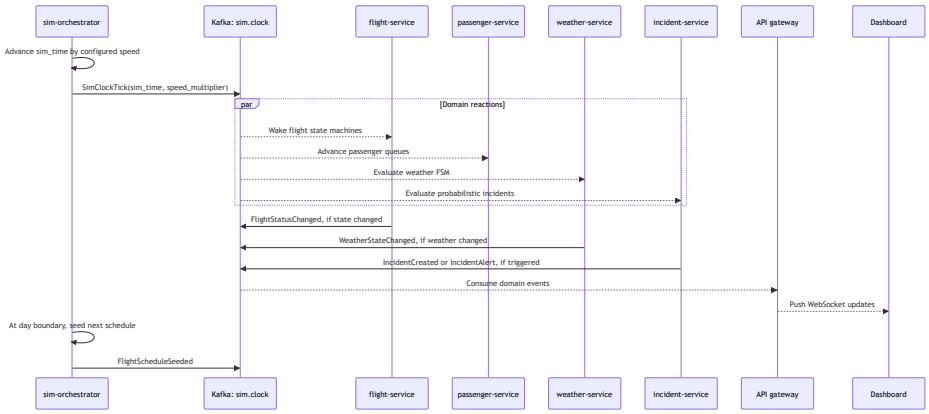


Fig. 7. Simulation-clock coordination in EVSK. A single virtual-time stream drives every service, and resulting facts are pushed to the dashboard through the gateway.

iThenticate Page 19 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 20 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

References

1. Glaessgen, E., Stargel, D.: The digital twin paradigm for future NASA and US air force vehicles. In: 53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference (2012)
2. Grieves, M., Vickers, J.: Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In: Transdisciplinary Perspectives on Complex Systems, pp. 85–113. Springer (2017)
3. Yang, C., Srirenganathan Malarvizhi, A., Masri, Y., Smith, J.S., Kim, J.: Digital twins as decision infrastructure: evolution, architecture, and research roadmap. Big Earth Data (2026), https://www.tandfonline.com/doi/full/10.1080/20964471.2026.2678046
4. AI Transformation Readiness: Changi airport — AI & digital transformation case study. https://www.aitransformationreadiness.org/post/changi-airport-digital-transformation (2026), accessed 19 August 2026
5. Jones, D.E., Snider, C., Nassehi, A., Yon, J.M., Hicks, B.J.: Characterising the digital twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology (2020), https://api.semanticscholar.org/CorpusID:216362509
6. International Organization for Standardization: ISO 23247-2:2021 automation systems and integration — digital twin framework for manufacturing — part 2: Reference architecture. Tech. rep., ISO (2021)
7. Adolphs, P., Bedenbender, H., Dirzus, D., Ehlich, M., Epple, U., Hankel, M., Wollschlaeger, M.: Reference architecture model Industrie 4.0 (RAMI4.0). Tech. rep., VDI/VDE and ZVEI (2015)
8. Redelinghuys, A.J., Basson, A.H., Kruger, K.: A six-layer architecture for the digital twin: A manufacturing case study implementation. Journal of Intelligent Manufacturing (2020)
9. Pyrgiotis, N.: Modelling delay propagation within an airport network. Transportation Research Part C-emerging Technologies 27, 60–75 (2013), https://api.semanticscholar.org/CorpusID:40671717
10. Cook, A., Tanner, G., Williams, V., Meise, G.: Dynamic cost indexing – managing airline delay costs. Journal of Air Transport Management 15(1), 26–35 (2009)
11. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: a highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
12. Schultz, M., Reitmann, S.: Machine learning approach to predict aircraft boarding. Transportation Research Part C: Emerging Technologies 98, 391–408 (2019)
13. Conde, J., Muñoz-Arcentales, A., Romero, M., Rojo, J., Salvachúa, J., Huecas, G., Alonso, Á.: Applying digital twins for the management of information in turnaround event operations in commercial airports. arXiv preprint arXiv:2408.14291 (2024)
14. Palaşcă, A., Semenescu, A., Stăncel, I.N.: Using digital twin in simulating airport landside operations. Annals of the Academy of Romanian Scientists Series on Engineering Sciences 17(2), 137–148 (2025)
15. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
16. Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M.: The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590 (2023)

iThenticate Page 20 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

iThenticate Page 21 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485

17. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: can language models resolve real-world GitHub issues? arXiv preprint arXiv:2310.06770 (2023)
18. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793 (2024)
19. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital twin in manufacturing: a categorical literature review and classification. In: 16th IFAC Symposium on Information Control Problems in Manufacturing (INCOM). IFAC-PapersOnLine, vol. 51, pp. 1016–1022 (2018)
20. Mohammadi, N., Taylor, J.E.: Smart city digital twins. 2017 IEEE Symposium Series on Computational Intelligence (SSCI) pp. 1–5 (2017), https://api.semanticscholar.org/CorpusID:44000518
21. Transportation Research Board: Guidebook for digital twin programs at airports (ACRP project 03-66) (2024)
22. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., et al.: MetaGPT: meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 (2023)
23. Lamport, L.: Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley (2002)
24. Parnas, D.L.: On the criteria to be used in decomposing systems into modules. Communications of the ACM 15(12), 1053–1058 (1972)
25. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)
26. Angles, R., Gutierrez, C.: Survey of graph database models. ACM Computing Surveys 40(1), 1–39 (2008)
27. EUROCONTROL: Standard inputs for EUROCONTROL cost-benefit analyses (2024), https://www.eurocontrol.int
28. European Parliament and Council: Regulation (EC) No 261/2004 establishing common rules on compensation and assistance to passengers in the event of denied boarding and of cancellation or long delay of flights (2004), official Journal of the European Union, L 46
29. DeMillo, R.A., Lipton, R.J., Sayward, F.G.: Hints on test data selection: help for the practicing programmer. Computer 11(4), 34–41 (1978)
30. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14, 131–164 (2009), https://api.semanticscholar.org/CorpusID:207144526

iThenticate Page 21 of 21 - Integrity Submission Submission ID trn:oid:::3117:617339485